

ProtejaJA — Documentação Técnica Completa

Plataforma web para registro de **denúncias** com cadastro de usuários, login, recuperação de senha e anexo de provas. Desenvolvida com **Python + Flask** (backend), **SQLite** (banco de dados) e **Tailwind CSS + CSS puro** (front-end).

1. Arquitetura geral

```
+-----+ HTTP (GET/POST) +-----+ SQL +-----+
| Navegador | -----> | Flask (Python) | -----> | SQLite |
| (Front-end) | ■----- | (Back-end) | ■----- | (app.db) |
| Tailwind + CSS | HTML / Redireciona | 16 rotas | | users/reports|
+-----+ +-----+ +-----+
```

- **Front-end:** páginas na pasta home/ + templates em backend/templates/, estilizadas com Tailwind (CDN) e home/css/styles.css.
 - **Back-end:** aplicação Flask em backend/app.py; recebe os formulários, valida, consulta/grava no banco e responde.
 - **Banco de dados:** arquivo db/app.db (SQLite). Tabelas users (usuários) e reports (denúncias).
 - **Conexão front↔back:** formulários HTML fazem POST para as rotas; o servidor responde com páginas renderizadas (server-side).
-

2. Estrutura de pastas

```
Default Project/
■--- home/ -> FRONT-END (pasta home)
| ■--- index.html -> página inicial estática
| ■--- css/styles.css -> tema rosa centralizado (CSS puro)
| +--- js/home.js -> preview de anexo + alertas
■--- backend/ -> BACK-END
| ■--- app.py -> aplicação Flask (todas as rotas)
| ■--- database.py -> conexão e operações com o SQLite
| ■--- profanity.py -> filtro antiofensa
| ■--- requirements.txt -> dependências (Flask)
| ■--- uploads/ -> provas anexadas (criada ao rodar)
| +--- templates/
| ■--- base.html -> layout base (nav + alertas)
| ■--- home.html -> página inicial servida pelo Flask
| ■--- cadastro.html -> formulário de cadastro
| ■--- login.html -> login + link recuperar senha
| ■--- recuperar.html -> recuperação de senha
| +--- denuncias.html -> form de denúncia + listagem
■--- db/app.db -> banco SQLite (gerado na 1ª execução)
■--- run.py -> inicia o servidor
■--- README.md -> guia rápido
+--- DOCUMENTACAO.md -> este documento
```

3. Explicação arquivo por arquivo

3.1 run.py — ponto de entrada

```
python
BASE_DIR = os.path.dirname(os.path.abspath(file))
sys.path.insert(0, os.path.join(BASE_DIR, "backend")) # permite importar o app
from app import create_app
app = create_app()
app.run(debug=True, host="127.0.0.1", port=5000)
```

O que faz: adiciona a pasta backend ao caminho de importação do Python, cria a aplicação Flask (corpo em app.py) e inicia o servidor em `http://127.0.0.1:5000`, com `debug=True` para recarregar sozinho ao editar.

3.2 backend/database.py — camada de dados

Função	O que faz
<code>get_connection()</code>	Abre conexão com <code>db/app.db</code> . <code>row_factory = sqlite3.Row</code> permite acessar colunas pelo nome (ex.: <code>linha["email"]</code>). Ativa <code>PRAGMA foreign_keys</code> para respeitar chaves estrangeiras.
<code>init_db()</code>	Cria as pastas <code>db/</code> e <code>backend/uploads/</code> , e as tabelas <code>users</code> e <code>reports</code> (se não existirem). É chamada na inicialização da aplicação.
<code>close_connection()</code>	Recuperação de contexto do Flask (usada no <code>teardown_appcontext</code>).
<code>email_cadastrado(email)</code>	Verifica se um e-mail já existe (colação <code>NOCASE</code> = ignora maiúsculas/minúsculas).
<code>criar_usuario(...)</code>	Insere um usuário no banco e devolve o id.
<code>usuario_por_email(email)</code>	Busca um usuário pelo e-mail (usado no login e na recuperação).
<code>usuario_por_id(id)</code>	Busca um usuário pelo id (usado na sessão).
<code>atualizar_senha(id, hash)</code>	Atualiza o hash da senha (recuperação de senha).
<code>criar_denuncia(...)</code>	Insere uma denúncia ligada ao <code>user_id</code> e devolve o id.
<code>denuncias_do_usuario(id)</code>	Lista as denúncias do usuário, da mais recente para a mais antiga.

Schema do banco:

```
sql
users:
id INTEGER PK AUTOINCREMENT
nome TEXT NOT NULL -> nome completo
email TEXT NOT NULL UNIQUE -> login
data_nascimento TEXT NOT NULL -> usada na recuperação de senha
cidade TEXT NOT NULL
endereço TEXT NOT NULL -> endereço completo
senha_hash TEXT NOT NULL -> senha SEMPRE criptografada
criado_em TEXT -> data/hora local

reports:
id INTEGER PK AUTOINCREMENT
user_id INTEGER NOT NULL -> FK para users.id
categoria TEXT NOT NULL DEFAULT 'Geral'
descricao TEXT NOT NULL
```

anexo TEXT -> nome do arquivo de prova
criado_em TEXT

Segurança no banco: todas as consultas usam **parâmetros ?** (nunca concatenação de texto), o que impede SQL Injection.

3.3 backend/profanity.py — filtro antiofensa

Função	O que faz
_sem_acentos(texto)	Remove acentos (normalização Unicode NFD). Assim "otário" e "otario" são tratados iguais.
_normalizar(texto)	Minúsculas + sem acentos + substitui caracteres "leet" (4->a, 3->e, 0->o, @->a, 1->i, 5->s, \$->s, 7->t, !->i, 2->z). Bloqueia tentativas tipo "c@r@lho".
contem_ofensa(texto)	Retorna True se o texto normalizado contém alguma palavra da lista de bloqueio (com limites de palavra \b, evitando falsos positivos tipo "cu" dentro de "acumular").
sanitizar(texto)	Se houver ofensa, substitui por **** e devolve o texto mascarado.

Lista de bloqueio: mais de 40 termos de baixo calão e ofensas em português, incluindo variações ("fdp", "filho da puta", "tomar no cu", etc.).

Onde é usado:

- Cadastro: bloqueia nome/cidade/endereço ofensivos.
- Denúncia: bloqueia descrição ofensiva e **sanitiza** antes de gravar.

3.4 backend/app.py — a aplicação Flask

Funções auxiliares (fora das rotas):

Função	O que faz
arquivo_permitido(nome)	Verifica se a extensão do anexo está na lista permitida (imagem, PDF, vídeo).
login_obrigatorio(visao)	Decorator: se não houver user_id na sessão, redireciona para o login com mensagem. Protege a página de denúncias.
validar_email(email)	Regex simples: algo@algo.algo.
usuario_atual()	Lê a sessão e devolve os dados do usuário logado (ou None).

Rotas (o coração do sistema):

Rota	Método	O que faz
/	GET	Redireciona para /home.
/home	GET	Renderiza a página inicial.
/home/	GET	Serve os arquivos front-end da pasta home/ (CSS, JS, index.html).
/uploads/	GET	Serve as provas anexadas.

Rota	Método	O que faz
/cadastro	GET/POST	GET : mostra o formulário. POST : valida campos, bloqueia ofensa, verifica e-mail duplicado, criptografa a senha* com generate_password_hash (bcrypt) e salva; depois redireciona para /login.
/login	GET/POST	POST : busca usuário por e-mail, confere a senha com check_password_hash, grava session["user_id"] e redireciona para /home.
/logout	GET	Encerra a sessão (session.clear()) e volta para /home.
/recuperar	GET/POST	POST : confere e-mail + data de nascimento; se bater, gera senha temporária (secrets.token_urlsafe(8)), criptografa e grava no banco, e mostra a senha na tela uma única vez.
/denuncias	GET/POST	Protegida por login . POST : valida descrição (obrigatória e sem ofensa), salva anexo (extensão permitida, nome seguro com secure_filename, máx. 5 MB em MAX_CONTENT_LENGTH), grava a denúncia. GET : lista as denúncias do usuário.

Mensagens ao usuário: o Flask usa flash("mensagem", "categoria") para feedback — categoria success (verde) ou error (vermelho/rosa) — exibidas no base.html.

4. Telas e template por template

4.1 base.html (pai de todas)

- Navbar fixa: logo **ProtejaJA**, links (Home, Minhas denúncias, Entrar/Sair, Cadastrar).
- Exibe as mensagens flash em alertas coloridos.
- Carrega o **Tailwind via CDN** (cdn.tailwindcss.com) e o **CSS da pasta home** (home/css/styles.css).
- Usa a variável user (fornecida pelo context_processor) para decidir se mostra "Entrar/Cadastrar" ou "Olá, nome + Sair".

4.2 home.html — página inicial

- Centralizada: título grande **ProtejaJA**, parágrafo explicando as funcionalidades.
- 3 cards (senha criptografada, filtro antiofensa, anexo de prova).
- Botões: se logado -> "Enviar denúncia"; se não -> "Criar conta" / "Entrar".

4.3 cadastro.html

- Campos: **nome completo, e-mail, data de nascimento, cidade, senha, endereço completo** (grid 2 colunas).
- Botão rosa escuro "Cadastrar" -> envia para /cadastro; ao conseguir, o servidor **redireciona para o login**.
- Link "Já tem conta? Entrar".

4.4 login.html

- Campos e-mail + senha; botão "Entrar".
- **Link visível "Recuperar senha"** (sublinhado, rosa) -> /recuperar.
- A senha é conferida com hash; se errada, mensagem "E-mail ou senha incorretos".

4.5 recuperar.html

- Formulário: e-mail + data de nascimento.
- Ao validar, exibe a **senha temporária** em destaque e o botão "Ir para o login".

4.6 denuncias.html

- Formulário: **categoria** (Geral/Local/Produto/Serviço/Segurança), **descrição** (obrigatória) e **anexo de prova** (imagem, PDF ou vídeo, até 5 MB).
- Lista "Minhas denúncias": categoria, data, texto e link para visualizar o anexo.

4.7 Pasta home/ (front-end)

- index.html — versão estática da home (abre pelo Flask em /home/index.html).
- css/styles.css — **tema rosa**: fundo gradiente rosa (#ffe4e6 -> #fecdd3), cards brancos com borda rosa, botões **rosa escuro** (#be123c), letras pretas (--slate-900), telas centralizadas.
- js/home.js — esconde alertas automaticamente após 6 s e mostra **pré-visualização do anexo** antes do envio.

5. Segurança implementada

1. **Senhas criptografadas**: generate_password_hash() (algoritmo scrypt, com salt) grava só o hash. O login usa check_password_hash(). **Nunca** há senha em texto puro no banco.
2. **Filtro antiofensa**: bloqueia xingamentos/ofensas em nome, cidade, endereço e descrição, inclusive com acentos e "leet speak".
3. **SQL Injection**: todas as consultas usam placeholders ? do sqlite3.
4. **Login obrigatório** para denunciar (@login_obrigatorio).
5. **Sessão segura**: session["user_id"] com secret_key.
6. **Upload seguro**: extensões permitidas, secure_filename, limite de 5 MB (MAX_CONTENT_LENGTH).
7. **Dados pessoais protegidos**: as denúncias só aparecem para o próprio usuário.

6. Fluxo do usuário (o que você vai apresentar)

1. Usuário abre /home
2. Sem conta -> "Criar conta" (cadastro: nome, nascimento, cidade, endereço, senha)
3. Cadastro válido -> redireciona para o LOGIN
4. Entra com e-mail + senha [ou "Recuperar senha" -> senha temporária]
5. Página /denúncias -> preenche categoria + descrição (+ prova anexada)
6. Denúncia salva (filtrada contra ofensas) e listada em "Minhas denúncias"
7. Sair -> sessão encerrada

7. Como rodar

```
powershell
cd "C:\Users\aline\OneDrive\Documentos\Default Project"
py -m venv .venv # só na primeira vez
```

```
..venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt # só na primeira vez
python run.py
```

Abrir: <http://127.0.0.1:5000/home>

8. Roteiro de explicação (para a banca/apresentação)

Fale nesta ordem: **problema -> solução -> telas -> código -> segurança.**

1. **Problema:** denúncias precisam ser feitas de forma segura, sem ofensas e com prova.
 2. **Solução:** site com cadastro, login seguro, recuperação de senha e anexo de prova.
 3. **Demonstre as telas:** cadastro -> login -> denúncia (mostre o anexo).
 4. **Mostre o código:** abra app.py e aponte:
 5. a rota /cadastro (validações e generate_password_hash na linha que criptografa a senha);
 6. a rota /denuncias (filtro + upload);
 7. profanity.py (lista de bloqueio e normalização);
 8. database.py (tabelas e consultas parametrizadas).
 9. **Segurança:** "senha nunca fica em texto puro" — abra o db/app.db com o **DB Browser for SQLite** e mostre o campo senha_hash ilegível.
 10. **Banco:** mostre as tabelas users e reports e relacione com os formulários.
-

9. Melhorias futuras (sugestões)

- Envio da senha temporária por e-mail real (biblioteca smtplib/flask-mail).
- Confirmação de e-mail e autenticação em duas etapas.
- Recuperação com pergunta de segurança ou link com validade.
- Página do professor/administrador para moderar denúncias.
- Deploy em produção (substituir servidor de desenvolvimento por Waitress/Gunicorn).